HPC Dotfiles and LMod

Rohit Goswami · 2020-08-09 · workflow · projects · hpc

Background

My move away from the powerful, but unimaginatively named HPC clusters of IITK [^fn:1] brought me in close contact with the Lua based [^fn:2] lmod module system. Rather than fall into the rabbit hole of brew we will leverage the existing system to add our new libraries. Not finding any good collections of these composable environments, and having failed once before to install Nix as a user without admin access, I decided to start my own collection of Lmod recipies. The rest of this post details the installation proceedure to be carried out in conjunction with the hzHPC_lmod repo.

Setting Up

These are reproduced from the repo for completeness.

```
git clone https://github.com/kobus-v-schoor/dotgit.git
mkdir -p ~/.local/bin
cp -r dotgit/old/bin/dotgit* ~/.local/bin
cat dotgit/old/bin/bash_completion >> ~/.bash_completion
rm -rf dotgit
```

I actually strongly suggest using a target from my Dotfiles in conjunction with this, but it isn't really required, so:

```
~/.local/bin/dotgit restore hzhpc
```

Note that because of the suggested separation, I have not opted to setup a shell or even ensure that there are scripts here to help keep `module` in your path. Those are in my Dotfiles. If, you **opt to not use** these `dotfiles`, then **do not** run the `ml load` commands.

Basic Setup

We will first load an environment with basic packages (`autotools`). This is a stop-gap solution.

MicroMamba

We will set up a `micromamba` installation to get `git` and other base utilities.

```
mkdir -p ~/.local/bin
export PATH=$HOME/.local/bin:$PATH
wget -qO- https://micromamba.snakepit.net/api/micromamba/linux-64/latest | tar -xvj bin/micromamba
mv bin/micromamba ~/.local/bin
rm -rf bin
micromamba shell init -s bash -p $HOME/micromamba
. ~/.bashrc
```

Now we can use `mamba` to get a temporary interactive environment

```
# Spack requirements
PKGS="git patch curl make tar gzip unzip bzip2 xz zstd subversion lsof m4"
mkdir ~/.mamba_venvs
micromamba create -p ~/.mamba_venvs/intBase $PKGS
micromamba activate ~/.mamba_venvs/intBase
```

For subsequent logins we can simply run:

```
export PATH=$HOME/.local/bin:$PATH
. ~/.bashrc
micromamba activate ~/.mamba_venvs/intBase
```

Configuration

A simple `.condarc` will suffice for the above.

```
channels:
  - conda-forge
  - defaults
channel_priority: disabled
```

Warning

Note that once the base packages have been installed...

> *We must **unload** our micromamba environment!!*

```
micromamba deactivate
```

It is recommended to use a minimal set of `micromamba` packages.

LMod Libraries

> ***Note that**: garpur and elja already has `lmod`*

The scripts in this post will also be part of the repo, but keep in mind that these are not meant to be reliable ways to install anything, and every command should be run by hand because things will probably break badly. We keep all sources in `$HOME/tmphpc` and install everything relative to `$HOME/.hpc`. Paths are managed by the `lmod` system.

```
export hpcroot=$HOME/tmphpc
mkdir -p $hpcroot
```

Combining our paths with the lmod manual gives rise to the following definiton (roughly the same for each one):

```
local home    = os.getenv("HOME")
local version = myModuleVersion()
local pkgName = myModuleName()
local pkg     = pathJoin(home,".hpc",pkgName,version,"bin")
prepend_path("PATH", pkg)
```

We will no longer bother with the module definitions for the rest of this post, as they are handled and documented in the repo.

```
# Setting up hzhpc modules
cd tmphpc
git clone https://github.com/kobus-v-schoor/dotgit.git
mkdir -p ~/.local/bin
cp -r dotgit/old/bin/dotgit* ~/.local/bin
cat dotgit/bin/bash_completion >> ~/.bash_completion
rm -rf dotgit
~/.local/bin/dotgit restore hzhpc
```

Patch

This is a basic utility we need before getting to anything else.

```
cd $hpcroot
myprefix=$HOME/.hpc/patch/2.7.2
wget http://ftp.gnu.org/gnu/patch/patch-2.7.2.tar.gz
gzip -dc patch-2.7.2.tar.gz | tar xvf -
cd patch-2.7.2
./configure --prefix=$myprefix
make -j$(nproc) && make install
ml load patch
```

Help2man

```
cd $hpcroot
myprefix=$HOME/.hpc/help2man/1.48.3
wget https://gnuftp.uib.no/help2man/help2man-1.48.3.tar.xz
tar xfv help2man-1.48.3.tar.xz
cd help2man-1.48.3
./configure --prefix=$myprefix
make -j$(nproc)
make install
ml load help2man/1.48.3
```

Perl

This is essentially the setup from the main docs.

```
# Get Perl
curl -L http://xrl.us/installperlnix | bash
cpanm --local-lib=~/perl5 local::lib && eval $(perl -I ~/perl5/lib/perl5/ -Mlocal::lib)
cpanm ExtUtils::MakeMaker --force # For git
cpanm Thread::Queue # For automake
ml load perl/5.28.0
```

Autotools

M4

This requires special considerations for `glibc` greater than `2.28` (true for compilers like `gcc8` and above).

```
cd $HOME/tmphpc
myprefix=$HOME/.hpc/autotools
wget http://ftp.gnu.org/gnu/m4/m4-1.4.18.tar.gz
gzip -dc m4-1.4.18.tar.gz | tar xvf -
cd m4-1.4.18
# From http://git.openembedded.org/openembedded-core/commit/meta/recipes-devtools/m4/m4/m4-1.4.18-glibc-change-work-around.patch
# From https://askubuntu.com/a/1112101/690387
wget http://git.openembedded.org/openembedded-core/plain/meta/recipes-devtools/m4/m4/m4-1.4.18-glibc-change-work-around.patch
patch  -p 1  < m4-1.4.18-glibc-change-work-around.patch
./configure -C --prefix=$myprefix/m4/1.4.18 && make -j$(nproc) && make install
```

Patch contents

The patch is needed since `glibc 2.28` and above have changed the nesting levels. The patch is effectively backported from the upstream repositories. Some more context is here.

```
update for glibc libio.h removal in 2.28+

see
https://src.fedoraproject.org/rpms/m4/c/814d592134fad36df757f9a61422d164ea2c6c9b?branch=master

Upstream-Status: Backport [https://git.savannah.gnu.org/cgit/gnulib.git/commit/?id=4af4a4a718]
Signed-off-by: Khem Raj <raj.khem@gmail.com>

Index: m4-1.4.18/lib/fflush.c
===================================================================
--- m4-1.4.18.orig/lib/fflush.c
+++ m4-1.4.18/lib/fflush.c
@@ -33,7 +33,7 @@
 #undef fflush

-#if defined _IO_ftrylockfile || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
+#if defined _IO_EOF_SEEN || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */

 /* Clear the stream's ungetc buffer, preserving the value of ftello (fp).  */
 static void
@@ -72,7 +72,7 @@ clear_ungetc_buffer (FILE *fp)

 #endif

-#if ! (defined _IO_ftrylockfile || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */)
+#if ! (defined _IO_EOF_SEEN || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */)

 # if (defined __sferror || defined __DragonFly__ || defined __ANDROID__) && defined __SNPT
 /* FreeBSD, NetBSD, OpenBSD, DragonFly, Mac OS X, Cygwin, Android */
@@ -148,7 +148,7 @@ rpl_fflush (FILE *stream)
   if (stream == NULL || ! freading (stream))
     return fflush (stream);

-#if defined _IO_ftrylockfile || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
+#if defined _IO_EOF_SEEN || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */

   clear_ungetc_buffer_preserving_position (stream);

Index: m4-1.4.18/lib/fpending.c
===================================================================
--- m4-1.4.18.orig/lib/fpending.c
+++ m4-1.4.18/lib/fpending.c
@@ -32,7 +32,7 @@ __fpending (FILE *fp)
   /* Most systems provide FILE as a struct and the necessary bitmask in
      <stdio.h>, because they need it for implementing getc() and putc() as
      fast macros.  */
-#if defined _IO_ftrylockfile || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
+#if defined _IO_EOF_SEEN || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
   return fp->_IO_write_ptr - fp->_IO_write_base;
 #elif defined __sferror || defined __DragonFly__ || defined __ANDROID__
   /* FreeBSD, NetBSD, OpenBSD, DragonFly, Mac OS X, Cygwin, Android */
Index: m4-1.4.18/lib/fpurge.c
===================================================================
--- m4-1.4.18.orig/lib/fpurge.c
+++ m4-1.4.18/lib/fpurge.c
@@ -62,7 +62,7 @@ fpurge (FILE *fp)
   /* Most systems provide FILE as a struct and the necessary bitmask in
      <stdio.h>, because they need it for implementing getc() and putc() as
      fast macros.  */
-# if defined _IO_ftrylockfile || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
+# if defined _IO_EOF_SEEN || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
   fp->_IO_read_end = fp->_IO_read_ptr;
   fp->_IO_write_ptr = fp->_IO_write_base;
   /* Avoid memory leak when there is an active ungetc buffer.  */
Index: m4-1.4.18/lib/freadahead.c
```

```
==================================================================
--- m4-1.4.18.orig/lib/freadahead.c
+++ m4-1.4.18/lib/freadahead.c
@@ -25,7 +25,7 @@
 size_t
 freadahead (FILE *fp)
 {
-#if defined _IO_ftrylockfile || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
+#if defined _IO_EOF_SEEN || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
   if (fp->_IO_write_ptr > fp->_IO_write_base)
     return 0;
   return (fp->_IO_read_end - fp->_IO_read_ptr)
Index: m4-1.4.18/lib/freading.c
==================================================================
--- m4-1.4.18.orig/lib/freading.c
+++ m4-1.4.18/lib/freading.c
@@ -31,7 +31,7 @@ freading (FILE *fp)
   /* Most systems provide FILE as a struct and the necessary bitmask in
      <stdio.h>, because they need it for implementing getc() and putc() as
      fast macros.  */
-# if defined _IO_ftrylockfile || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
+# if defined _IO_EOF_SEEN || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
   return ((fp->_flags & _IO_NO_WRITES) != 0
           || ((fp->_flags & (_IO_NO_READS | _IO_CURRENTLY_PUTTING)) == 0
               && fp->_IO_read_base != NULL));
Index: m4-1.4.18/lib/fseeko.c
==================================================================
--- m4-1.4.18.orig/lib/fseeko.c
+++ m4-1.4.18/lib/fseeko.c
@@ -47,7 +47,7 @@ fseeko (FILE *fp, off_t offset, int when
 #endif

   /* These tests are based on fpurge.c.  */
-#if defined _IO_ftrylockfile || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
+#if defined _IO_EOF_SEEN || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
   if (fp->_IO_read_end == fp->_IO_read_ptr
       && fp->_IO_write_ptr == fp->_IO_write_base
       && fp->_IO_save_base == NULL)
@@ -123,7 +123,7 @@ fseeko (FILE *fp, off_t offset, int when
           return -1;
         }

-#if defined _IO_ftrylockfile || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
+#if defined _IO_EOF_SEEN || __GNU_LIBRARY__ == 1 /* GNU libc, BeOS, Haiku, Linux libc5 */
       fp->_flags &= ~_IO_EOF_SEEN;
       fp->_offset = pos;
 #elif defined __sferror || defined __DragonFly__ || defined __ANDROID__
Index: m4-1.4.18/lib/stdio-impl.h
==================================================================
--- m4-1.4.18.orig/lib/stdio-impl.h
+++ m4-1.4.18/lib/stdio-impl.h
@@ -18,6 +18,12 @@
    the same implementation of stdio extension API, except that some fields
    have different naming conventions, or their access requires some casts.  */

+/* Glibc 2.28 made _IO_IN_BACKUP private.  For now, work around this
+   problem by defining it ourselves.  FIXME: Do not rely on glibc
+   internals.  */
+#if !defined _IO_IN_BACKUP && defined _IO_EOF_SEEN
+# define _IO_IN_BACKUP 0x100
+#endif

 /* BSD stdio derived implementations.  */
```

Auto{Conf,Make} and Libtool

This follows the standard approach outlined in the GNU Autotools FAQ:

```
cd $hpcroot
myprefix=$HOME/.hpc/autotools
export PATH
wget http://ftp.gnu.org/gnu/autoconf/autoconf-2.69.tar.gz
wget http://ftp.gnu.org/gnu/automake/automake-1.16.2.tar.gz
wget http://ftp.gnu.org/gnu/libtool/libtool-2.4.6.tar.gz
wget http://ftp.gnu.org/gnu/gettext/gettext-0.20.tar.gz
wget https://gnuftp.uib.no/autoconf-archive/autoconf-archive-2021.02.19.tar.xz
tar xfv autoconf-archive-2021.02.19.tar.xz
gzip -dc autoconf-2.69.tar.gz | tar xvf -
gzip -dc automake-1.16.2.tar.gz | tar xvf -
gzip -dc libtool-2.4.6.tar.gz | tar xvf -
gzip -dc gettext-0.20.tar.gz | tar xvf -
cd autoconf-2.69
./configure -C --prefix=$myprefix/autoconf/2.69 && make -j$(nproc) && make install
cd ../automake-1.16.2
./configure -C --prefix=$myprefix/automake/1.16.2 --docdir=$myprefix/automake/1.16.2/share/doc/automake-1.16.2
&& make -j$(nproc) && make install
cd ../autoconf-archive-2021.02.19
./configure -C --prefix=$myprefix/automake/1.16.2
cd ../libtool-2.4.6
./configure -C --disable-static --prefix=$myprefix/libtool/2.4.6 && make -j$(nproc) && make install
cd ../gettext-0.20
./configure -C --prefix=$myprefix/gettext/0.20 && make -j$(nproc) && make install
ml load autotools/autotools
```

GMP

```
cd $hpcroot
myprefix=$HOME/.hpc/gcc/gmp/6.2.0
export PATH
wget https://gmplib.org/download/gmp/gmp-6.2.0.tar.xz
tar xfv gmp-6.2.0.tar.xz
cd gmp-6.2.0
./configure --prefix=$myprefix    \
            --enable-cxx     \
            --docdir=$myprefix/doc/gmp-6.2.0
make -j$(nproc)
make install
ml load gcc/gmp/6.2.0
```

MPFR

```
cd $hpcroot
myprefix=$HOME/.hpc/gcc/mpfr/4.1.0
export PATH
wget https://www.mpfr.org/mpfr-current/mpfr-4.1.0.tar.xz
tar xfv mpfr-4.1.0.tar.xz
cd mpfr-4.1.0
./configure --prefix=$myprefix    \
            --enable-thread-safe     \
            --with-gmp=$HOME/.hpc/gcc/gmp/6.2.0 \
            --docdir=$myprefix/doc/mpfr-4.1.0
make -j$(nproc)
make install
ml load gcc/mpfr/4.1.0
```

MPC

```
cd $hpcroot
myprefix=$HOME/.hpc/gcc/mpc/1.2.0
export PATH
wget https://ftp.gnu.org/gnu/mpc/mpc-1.2.0.tar.gz
tar xfv mpc-1.2.0.tar.gz
cd mpc-1.2.0
./configure --prefix=$myprefix    \
            --with-gmp=$HOME/.hpc/gcc/gmp/6.2.0 \
            --with-mpfr=$HOME/.hpc/gcc/mpfr/4.1.0 \
            --docdir=$myprefix/doc/mpc-1.2.0
make -j$(nproc)
make install
ml load gcc/mpc/1.2.0
```

GCC 9.2.0

```
cd $hpcroot
ml load gcc/gmp gcc/mpfr gcc/mpc
myprefix=$HOME/.hpc/gcc/9.2.0
wget https://ftp.gnu.org/gnu/gcc/gcc-9.2.0/gcc-9.2.0.tar.xz
tar xfv gcc-9.2.0.tar.xz
cd gcc-9.2.0
case $(uname -m) in
  x86_64)
    sed -e '/m64=/s/lib64/lib/' \
        -i.orig gcc/config/i386/t-linux64
  ;;
esac
mkdir -p build                                          &&
cd    build                                             &&

SED=sed                                \
../configure --prefix=$myprefix            \
             --enable-languages=c,c++,fortran \
             --disable-multilib       \
             --with-gmp=$HOME/.hpc/gcc/gmp/6.2.0 \
             --with-mpfr=$HOME/.hpc/gcc/mpfr/4.1.0 \
             --with-mpc=$HOME/.hpc/gcc/mpc/1.2.0 \
             --disable-bootstrap      \
             --with-system-zlib
export PATH
unset LIBRARY_PATH
export LIBRARY_PATH=/usr/lib64/
mkdir -p -- .deps
make -j$(nproc)
make install
ml load gcc/9.2.0
```

Pkg-Config

```
cd $hpcroot
myprefix=$HOME/.hpc/pkg-config/0.29.2
wget https://pkg-config.freedesktop.org/releases/pkg-config-0.29.2.tar.gz
gzip -dc pkg-config-0.29.2.tar.gz | tar xvf -
cd pkg-config-0.29.2
./configure --prefix=$myprefix --with-internal-glib --disable-host-tool --docdir=$myprefix/share/doc/pkg-
config-0.29.2
mkdir $myprefix/lib
make -j $(nproc)
make install
ml load pkg-config/0.29.2
```

Zlib

```
cd $hpcroot
myprefix=$HOME/.hpc/zlib/1.2.11
wget http://zlib.net/zlib-1.2.11.tar.gz
gzip -dc zlib-1.2.11.tar.gz | tar xvf -
cd zlib-1.2.11
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load zlib/1.2.11
```

XZ Utils

```
cd $hpcroot
myprefix=$HOME/.hpc/xz/5.2.5
wget https://tukaani.org/xz/xz-5.2.5.tar.gz
gzip -dc xz-5.2.5.tar.gz | tar xvf -
cd xz-5.2.5
./configure --prefix=$myprefix --enable-threads=yes
make -j $(nproc)
make install
ml load xz/5.2.5
```

OpenSSL

```
cd $hpcroot
myprefix=$HOME/.hpc/openssl/1.1.1d
wget https://www.openssl.org/source/openssl-1.1.1d.tar.gz
gzip -dc openssl-1.1.1d.tar.gz | tar xvf -
cd openssl-1.1.1d
./config --prefix=$myprefix --openssldir=$myprefix/etc/ssl shared zlib-dynamic
make -j $(nproc)
make install
ml load openssl/1.1.1d
```

cURL

```
cd $hpcroot
myprefix=$HOME/.hpc/curl/7.76.0
wget https://curl.haxx.se/download/curl-7.76.0.tar.xz
tar xfv  curl-7.76.0.tar.xz
cd curl-7.76.0
./configure --prefix=$myprefix \
    --disable-static \
    --enable-threaded-resolver \
    --with-openssl
make -j $(nproc)
make install
ml load curl/7.76.0
```

Git

This is very similar to the previous approach. However, since by default the system `perl` was being picked up, some slight changes have been made.

```
cd $hpcroot
myprefix=$HOME/.hpc/git/2.9.5
PATH=$myprefix/bin:$PATH
export PATH
wget https://mirrors.edge.kernel.org/pub/software/scm/git/git-2.9.5.tar.gz
gzip -dc git-2.9.5.tar.gz | tar xvf -
cd git-2.9.5
./configure --with-perl=$(which perl) --with-curl=$(which curl) -C --prefix=$myprefix
make -j $(nproc)
make install
ml load git/2.9.5
ml unload openssl # System will suffice
ml unload curl # System will suffice
```

Caveat

Also, for TRAMP, we would prefer having a more constant path, so we can set up a symlink:

```
mkdir ~/.hpc/bin
ln ~/.hpc/git/2.9.5/bin/git ~/.hpc/bin/git
```

Boost

The boost website is utterly incomprehensible. As is the documentation. Also, fun fact, the move from `svn` makes things worse. This is best installed with the standard compiler present, since the B2 engine detection seems to be very hit and miss. Thankfully, a quick dive into the slightly better Github wiki led to this nugget:

```
cd $hpcroot
git clone --recursive https://github.com/boostorg/boost.git
cd boost
git checkout tags/boost-1.76.0 # or whatever branch you want to use
./bootstrap.sh
./b2 headers
```

This means we're almost done!

```
./b2
./b2 install --prefix=$HOME/.hpc/boost/boost-1.76.0
ml load boost/boost-1.76.0
```

CMake

```
cd $hpcroot
myprefix=$HOME/.hpc/cmake/3.20.1
wget https://github.com/Kitware/CMake/releases/download/v3.20.1/cmake-3.20.1.tar.gz
gzip -dc cmake-3.20.1.tar.gz | tar xvf -
cd cmake-3.20.1
CXXFLAGS="-std=c++11" CC=$(which gcc) CXX=$(which g++) ./bootstrap --prefix=$myprefix --system-libs
make -j $(nproc)
make install
ml load cmake/3.20.1
```

GNU-Make

```
cd $hpcroot
myprefix=$HOME/.hpc/make/4.3
wget http://ftp.gnu.org/gnu/make/make-4.3.tar.gz
gzip -dc make-4.3.tar.gz | tar xvf -
cd make-4.3
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load make/4.3
```

Brotli

```
cd $hpcroot
myprefix=$HOME/.hpc/brotli/1.0.1
git clone https://github.com/bagder/libbrotli
cd libbrotli
libtoolize
aclocal
autoheader
./autogen.sh
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load brotli/1.0.1
```

ncurses

We will need to manually ensure the paths for `pkg-config` are in a feasible location.

```
cd $hpcroot
myprefix=$HOME/.hpc/ncurses/6.2
wget https://invisible-mirror.net/archives/ncurses/ncurses-6.2.tar.gz
gzip -dc ncurses-6.2.tar.gz | tar xvf -
cd ncurses-6.2
./configure --prefix=$myprefix --enable-widec --enable-pc-files --with-shared
make -j $(nproc)
make install
mkdir pkgconfig
cp misc/formw.pc misc/menuw.pc misc/ncurses++w.pc misc/ncursesw.pc misc/panelw.pc pkgconfig/
mv pkgconfig $myprefix/lib/
ml load ncurses/6.2
```

texinfo

```
cd $hpcroot
myprefix=$HOME/.hpc/texinfo/6.7
wget http://ftp.gnu.org/gnu/texinfo/texinfo-6.7.tar.gz
gzip -dc texinfo-6.7.tar.gz | tar xvf -
cd texinfo-6.7
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load texinfo/6.7
```

`gperf`

```
cd $hpcroot
myprefix=$HOME/.hpc/gperf/3.1
wget http://ftp.gnu.org/gnu/gperf/gperf-3.1.tar.gz
gzip -dc gperf-3.1.tar.gz | tar xvf -
cd gperf-3.1
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load gperf/3.1
```

libseccomp

There is a bug, which requires modifying `src/system.c` to change `__NR_seccomp` to `_nr_seccomp`.

```
cd $hpcroot
myprefix=$HOME/.hpc/libseccomp/2.5.0
git clone https://github.com/seccomp/libseccomp
cd libseccomp
git checkout tags/v2.5.0
./autogen.sh
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load libseccomp/2.5.0
```

Alternatively, it is easier to work with an older version.

```
cd $hpcroot
myprefix=$HOME/.hpc/libseccomp/2.4.4
wget https://github.com/seccomp/libseccomp/releases/download/v2.4.4/libseccomp-2.4.4.tar.gz
tar xfv libseccomp-2.4.4.tar.gz
cd libseccomp-2.4.4
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load libseccomp/2.4.4
```

BDWGC

```
cd $hpcroot
myprefix=$HOME/.hpc/bdwgc/8.0.4
wget https://github.com/ivmai/bdwgc/releases/download/v8.0.4/gc-8.0.4.tar.gz
gzip -dc gc-8.0.4.tar.gz | tar xvf -
cd gc-8.0.4
./configure --prefix=$myprefix --enable-cplusplus
make -j $(nproc)
make install
ml load bdwgc/8.0.4
```

pcre

We will prep both `pcre2` and `pcre`.

```
cd $hpcroot
myprefix=$HOME/.hpc/pcre2/10.35
wget https://ftp.pcre.org/pub/pcre/pcre2-10.35.tar.gz
gzip -dc pcre2-10.35.tar.gz | tar xvf -
cd pcre2-10.35
./configure --prefix=$myprefix \
            --enable-pcre2-16  \
            --enable-pcre2-32  \
            --enable-pcre2grep-libz
make -j $(nproc)
make install
ml load pcre2/10.35
```

```
cd $hpcroot
myprefix=$HOME/.hpc/pcre/8.44
wget https://ftp.pcre.org/pub/pcre/pcre-8.44.tar.gz
gzip -dc pcre-8.44.tar.gz | tar xvf -
cd pcre-8.44
./configure --prefix=$myprefix \
            --enable-pcre-16  \
            --enable-pcre-32  \
            --enable-pcregrep-libz
make -j $(nproc)
make install
ml load pcre/8.44
```

bison

```
cd $hpcroot
myprefix=$HOME/.hpc/bison/3.7.1
wget http://ftp.gnu.org/gnu/bison/bison-3.7.1.tar.gz
gzip -dc bison-3.7.1.tar.gz | tar xvf -
cd bison-3.7.1
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load bison/3.7.1
```

`flex`

```
cd $hpcroot
myprefix=$HOME/.hpc/flex/2.6.4
wget https://github.com/westes/flex/releases/download/v2.6.4/flex-2.6.4.tar.gz
gzip -dc flex-2.6.4.tar.gz | tar xvf -
cd flex-2.6.4
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load flex/2.6.4
```

`jq`

We first need the `oniguruma` regular expression library.

```
cd $hpcroot
myprefix=$HOME/.hpc/oniguruma/6.9.7.1
git clone git@github.com:kkos/oniguruma.git
cd oniguruma
git checkout tags/v6.9.7.1
libtoolize
autoreconf -i
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load oniguruma/6.9.7.1
```

We will actually use the binary.

```
cd $HOME/.local/bin
wget https://github.com/stedolan/jq/releases/download/jq-1.6/jq-linux64
chmod +x jq-linux64
mv jq-linux64 jq
```

bzip2

Needed to manually configure it as shown here

```
cd $hpcroot
myprefix=$HOME/.hpc/bzip2/1.0.8
wget https://www.sourceware.org/pub/bzip2/bzip2-1.0.8.tar.gz
gzip -dc bzip2-1.0.8.tar.gz | tar xvf -
cd bzip2-1.0.8
make -f Makefile-libbz2_so
ln -sf libbz2.so.1.0 libbz2.so
mkdir -p $myprefix/include
mkdir -p $myprefix/lib
cp -avf bzlib.h $myprefix/include
cp -avf libbz2.so* $myprefix/lib
make install PREFIX=$myprefix
ml load bzip2/1.0.8
```

Actually there are a set of patches for `1.0.6` which include `pkg-config` support so we will use those as well.

```
cd $hpcroot
myprefix=$HOME/.hpc/bzip2/1.0.6
wget ftp://sourceware.org/pub/bzip2/bzip2-1.0.6.tar.gz
tar xfz bzip2-1.0.6.tar.gz
cd bzip2-1.0.6
# patches.list: https://gist.github.com/steakknife/0ee85c93495ab9f9cff5e21ee12fb25b
wget https://gist.githubusercontent.com/steakknife/946f6ee331512a269145b293cbe898cc/raw/bzip2-1.0.6-
install_docs-1.patch
wget https://gist.githubusercontent.com/steakknife/eceda09cae0cdb4900bcd9e479bab9be/raw/bzip2recover-CVE-2016-
3189.patch
wget https://gist.githubusercontent.com/steakknife/42feaa223adb4dd7c5c85f288794973c/raw/bzip2-man-page-
location.patch
wget https://gist.githubusercontent.com/steakknife/94f8aa4bfa79a3f896a660bf4e973f72/raw/bzip2-shared-make-
install.patch
wget https://gist.githubusercontent.com/steakknife/4faee8a657db9402cbeb579279156e84/raw/bzip2-pkg-config.patch
patch -u < bzip2-1.0.6-install_docs-1.patch
patch -u < bzip2recover-CVE-2016-3189.patch
patch -u < bzip2-man-page-location.patch
patch -u < bzip2-shared-make-install.patch
patch -u < bzip2-pkg-config.patch
make
make install PREFIX=$myprefix
ml load bzip2/1.0.6
```

sqlite

```
cd $hpcroot
myprefix=$HOME/.hpc/sqlite/3.32.3
wget https://www.sqlite.org/2020/sqlite-autoconf-3320300.tar.gz
gzip -dc sqlite-autoconf-3320300.tar.gz | tar xvf -
cd sqlite-autoconf-3320300
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load sqlite/3.32.3
```

editline

```
cd $hpcroot
myprefix=$HOME/.hpc/editline/1.17.1
wget https://github.com/troglobit/editline/releases/download/1.17.1/editline-1.17.1.tar.gz
gzip -dc editline-1.17.1.tar.gz | tar xvf -
cd editline-1.17.1
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load editline/1.17.1
```

Miniconda

We don’t need this very much, but it is still useful for some edge cases, mainly revolving around `jupyter` infrastructure.

```
cd $HOME
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
chmod +x Miniconda3-latest-Linux-x86_64.sh
./Miniconda3-latest-Linux-x86_64.sh
# Do not allow it to mess up the shell rc files
eval "$($HOME/miniconda3/bin/conda shell.zsh hook)"
```

Note that we will prefer the manual evaluation since it can be handled in the `lmod` file.

Applications

Libraries and `git` aside, there are some tools we might want to have.

ag

The silver searcher, along with `rg` is very useful to have.

```
cd $hpcroot
myprefix=$HOME/.hpc/the_silver_searcher/2.2.0
wget https://geoff.greer.fm/ag/releases/the_silver_searcher-2.2.0.tar.gz
gzip -dc the_silver_searcher-2.2.0.tar.gz | tar xvf -
cd the_silver_searcher-2.2.0
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load the_silver_searcher/2.2.0
```

Neovim

```
cd $hpcroot
myprefix=$HOME/.hpc/nvim/0.5.0
wget https://github.com/neovim/neovim/releases/download/nightly/nvim.appimage
chmod +x nvim.appimage
./nvim.appimage --appimage-extract
mkdir -p $myprefix
mv squashfs-root/usr/* $myprefix
ml load nvim/0.5.0
```

Tmux

```
cd $hpcroot
myprefix=$HOME/.hpc/tmux/3.1b
wget https://github.com/tmux/tmux/releases/download/3.1b/tmux-3.1b-x86_64.AppImage
chmod +x tmux-3.1b-x86_64.AppImage
rm -rf squashfs-root
./tmux-3.1b-x86_64.AppImage --appimage-extract
mkdir -p $myprefix
mv squashfs-root/usr/bin squashfs-root/usr/lib squashfs-root/usr/share $myprefix
ml load tmux/3.1b
```

Zsh

More of an update than a requirement.

```
cd $hpcroot
myprefix=$HOME/.hpc/zsh/5.8
wget https://github.com/zsh-users/zsh/archive/zsh-5.8.tar.gz
gzip -dc zsh-5.8.tar.gz | tar xvf -
cd zsh-zsh-5.8
autoreconf -fiv
./configure --prefix=$myprefix
make -j $(nproc)
make install
ml load zsh/5.8
```

Conclusion

Having composed a bunch of these, I will of course try to somehow get `nix` up and running so it can bootstrap itself and allow me to work in peace. I might also eventually create shell scripts to automate updating these, but hopefully I can set up `nix` and not re-create package manager logic in `lua`.
